

Screenshot Clipboard Harness: Claude Code Handoff

Oct 8, 2026 · @Mehmet Sat

Goal

This run tests whether a planner → generator → evaluator harness works on a macOS app, and whether we can watch which agent is doing what, live. The app itself is secondary; what we measure is the loop, the handoff files and the observability layer.

Reference: Anthropic's [Harness design for long-running application development](#) and [Effective harnesses for long-running agents](#).

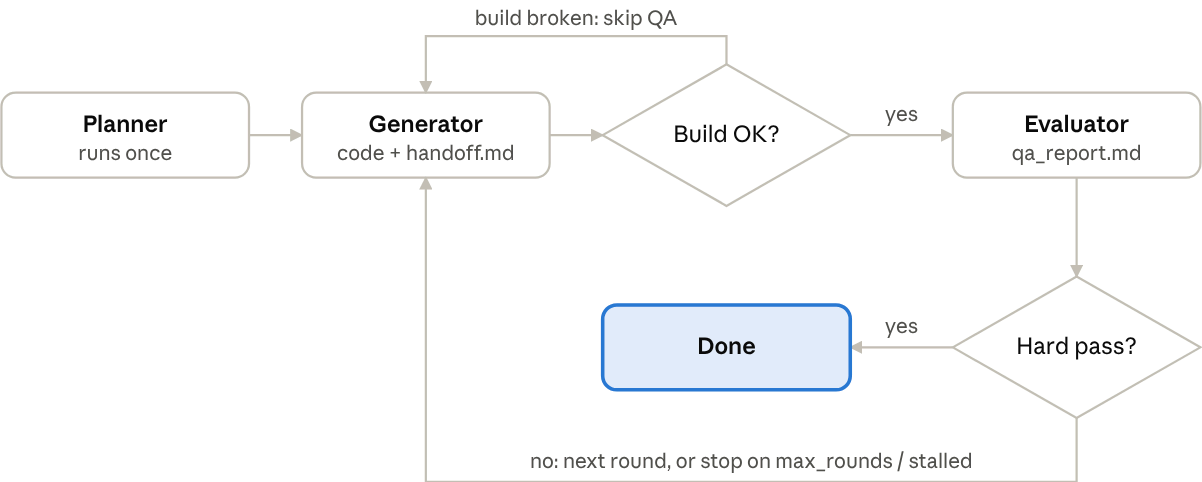
Success for this first run:

- The loop completes at least 3 rounds without the orchestrator crashing
- The evaluator writes at least 5 concrete, actionable findings per round
- The panel shows at any moment which agent is running which tool
- Token and cost breakdown per agent per round is produced
- The final build launches, captures screenshots, shows them in the grid

Harness architecture

Three agents, one orchestrator, communication through files. Control flow is a deterministic loop in orchestrator code; agents never trigger each other. No sprint construct: the planner runs once, then generator ↔ evaluator alternate.

QA feedback loops back to the generator, never to the planner



harness loop · 3 agents, 2 decisions

A broken build goes straight back to the generator without a QA pass; a QA fail repeats the round until every hard criterion passes or a stop rule fires.

Agent	Reads	Writes	Tools
Planner (once)	prompt.md + constraints	spec.md	read, write
Generator (each round)	spec.md , qa_report.md (round ≥2), git log	code, handoff.md , git commit	bash, read, write, edit, build.sh , run.sh
Evaluator (each round)	spec.md , handoff.md , the running app	qa_report.md (JSON + markdown)	bash, read, ax_tree , screenshot , record_frames

The loop:

1. Planner writes `spec.md` . Orchestrator parses the criteria list out of it.
2. Generator round N: reads spec and the previous `qa_report.md` if any, closes only the failing items, builds, self-tests, writes `handoff.md` , commits.
3. Orchestrator runs `build.sh` . Broken build → straight back to generator, evaluator is skipped.
4. Evaluator launches the app via `run.sh` , walks every criterion with AX tree + screenshots, writes `qa_report.md` .

5. Orchestrator reads the report: all hard criteria pass → stop, otherwise round N+1.

Termination (all in orchestrator code):

- All hard criteria pass → done
- `max_rounds` reached (5 for the first run) → done, status "incomplete"
- Hard pass count did not increase for two consecutive rounds → done, status "stalled"
- Evaluator sets `spec_issue: true` → stop and ask a human (no automatic return to planner)

Criterion classes:

- **Hard** (functionality, craft): any fail repeats the round
- **Soft** (design, motion): produces a score and feedback, never fails a round on its own
- **Manual**: the evaluator cannot test it (drag-out to other apps, hotkey while another app is focused); stays marked "manual" in the report

Test app: screenshot clipboard

A menubar macOS app that keeps every screenshot you take, shows them in a grid, and lets you search, pin and re-copy them. Screenshots only; text and other clipboard content is ignored. The planner expands this into `spec.md`; these constraints go into `prompt.md` verbatim.

Constraints for the planner:

- Captures screenshots from `⌘↑3` / `⌘↑4` / `⌘↑5` and any image copied to the pasteboard. Nothing else.
- Menubar icon plus a separate window. The window shows a thumbnail grid with date and size, search over filename and OCR text, pin, delete.
- Every capture is written to disk; metadata in SQLite. Quit and relaunch, everything is still there.
- Click copies back to the pasteboard, double-click opens in Preview, drag out to other apps.
- Global hotkey opens the window.
- OCR via the Vision framework for searchable text. This is the app's "AI feature"; no model API calls.
- Retention setting: keep last N captures or last N days.

- Stack: Swift, SwiftUI, SQLite (GRDB or raw sqlite3), macOS 14+. Built with `xcodebuild`, no third-party UI frameworks.

Motion, as three named hero interactions (soft criteria):

- Grid → full preview: the selected card expands into the preview (`matchedGeometryEffect`), background blurs.
- Preview navigation: arrow keys or swipe slide to the neighbour like a slider; current image scales down slightly as it leaves.
- New capture: enters at the head of the grid with a slide, the rest make room. Pin and delete animate the card out of place.

Known friction, decided up front:

- Screen Recording and Accessibility permissions are granted by hand once during init and noted in `spec.md`; agents must not try to work around them.
- Drag-out and hotkey-from-another-app are manual criteria.
- Target 25 to 30 hard criteria in the spec. If the planner produces more than 40, trim before round 1.

Evaluator tooling for a Mac app

There is no Playwright for macOS, so the evaluator gets four small scripts exposed as tools. Build these before the first run; they are the part of the harness most likely to be wrong.

Tool	What it does	Implementation
<code>ax_tree [window]</code>	Dumps the accessibility tree of the app window as JSON: role, title, value, frame, enabled	Swift CLI using <code>AXUIElement</code> ; needs Accessibility permission for the terminal
<code>ax_click <id> /</code> <code>ax_type <id> <text> /</code> <code>ax_key <combo></code>	Performs an action on an AX element or sends a keystroke	Same CLI, <code>AXUIElementPerformAction</code> + <code>CGEvent</code>
<code>screenshot [window]</code>	PNG of the app window only, returned to the model as an image	<code>screencapture -l <windowID>;</code> window id via <code>CGWindowListCopyWindowInfo</code>

Tool	What it does	Implementation
<code>record_frames</code> <code><seconds> <n></code>	Records the window for N seconds and returns N evenly spaced frames	<code>screencapture -V</code> then <code>ffmpeg -vf fps=</code>

Rules for the evaluator prompt:

- Verify visual content (thumbnails, previews, animation) with `screenshot` or `record_frames`. The AX tree shows an image as one node and says nothing about its content.
- Trigger a real capture during the test: `screencapture -x /tmp/t.png` then copy it to the pasteboard with a small `pbcopy` -style helper, or send `⌘↑3` via `ax_key`.
- Motion criteria: call `record_frames 2 8` around the interaction, then judge "no transition / broken / smooth" from the frames. Easing and timing taste stays with the human.

Build and run wrappers for the generator:

- `build.sh`: `xcodebuild` piped through `xcbeautify` (or a `grep` for `error:` and `warning:`), exit code preserved. Raw `xcodebuild` output is noisy enough to pollute context and cost input tokens every turn.
- `run.sh`: kills any running instance, launches the built `.app`, waits until the window is up, prints the window id.
- `init.sh`: one-time setup. Creates the Xcode project, grants permissions (manual step, script pauses), seeds a few test screenshots.

Prerequisites on the machine: Xcode 16+, `xcbeautify`, `ffmpeg`, Accessibility + Screen Recording for the terminal running the harness.

Observability

One event stream feeds everything. The orchestrator normalises every Agent SDK stream message into one event shape and appends it to `runs/<run_id>/events.jsonl`; the same events go into SQLite for queries. Observability only reads; it never makes control decisions.

Event shape:

```
{
  "ts": "2026-10-08T14:02:11Z",
  "run_id": "r_2026-10-08_a",
  "agent": "generator",
  "round": 2,
  "type": "tool_use",
  "tool": "bash",
  "summary": "./build.sh",
  "duration_ms": 201400,
  "input_tokens": 48210,
  "output_tokens": 1120,
  "cost_usd": 0.0106,
  "payload": {}
}
```

Event types: `agent_start`, `agent_end`, `assistant_text`, `tool_use`, `tool_result`, `artifact_written` (from an `fswatch` on the handoff directory), `build_result`, `qa_verdict`, `round_start`, `round_end`, `error`. Keep `payload` truncated to 2 KB in the JSONL; full tool output goes to `runs/<run_id>/raw/`.

Live panel (first version: a single HTML page served by the orchestrator, SSE from the JSONL tail):

- One lane per agent: state (idle / thinking / tool), current tool with elapsed time, last three tool calls, first line of the last assistant message
- Round counter, running token and cost totals per agent, context fill as a percentage of 1M (count with the new tokenizer, roughly 30% more tokens than Haiku 4.5 for the same text)
- Artifacts pane: `spec.md`, `handoff.md`, `qa_report.md` with diff against the previous round
- Screenshot gallery from the evaluator, newest first
- Hard criteria pass count per round as a small bar

Traces for after-the-fact analysis: wrap each round in a span and each tool call in a child span with OpenTelemetry, export to a self-hosted Langfuse or Phoenix. Not needed for run 1; the JSONL is enough to answer "why did QA pass this". Add it once the loop is stable.

Quick-and-dirty fallback for day one: each agent in its own tmux pane with `tee` to a log file. Gives signal, no cost or timing data.

Models, auth and cost

Build and debug the harness on the Team plan. Run the first real loops on Haiku 5.5 as generator with a stronger evaluator. Switch to an API key with a spending limit for long Opus runs.

Phase	Generator	Evaluator	Planner	Auth
Harness dev, short rounds	Haiku 5.5	Haiku 5.5	Haiku 5.5	Team seat (Claude Code login)
Run 1, full loop	Haiku 5.5	Sonnet 5.5	Sonnet 5.5	Team seat, watch the 5-hour window
Run 2, quality comparison	Opus 5.5	Sonnet 5.5 or Opus 5.5	Opus 5.5	API key, spending limit set

Why mix: in Anthropic's reference run the build rounds cost about \$70 each and QA rounds \$3 to \$4. The generator is the expensive agent; a strong evaluator is nearly free and is where the signal comes from. A weak generator with a strict evaluator is the best configuration for testing the loop itself.

Model facts as of today ([source](#)):

- Haiku 5.5: 1M context, 128K max output, from \$0.10 / \$0.50 per MTok, model id `claude-haiku-5-5`
- Sonnet 5.5: 1M context, \$2 / \$10 per MTok
- Opus 5.5: 1M context, \$4 / \$20 per MTok

Subscription note ([source](#)): Agent SDK and `claude -p` usage currently draws from the subscription's usage limits; the separate monthly SDK credit announced for June 15 is paused. Advertised limits assume ordinary individual use, so a multi-hour autonomous run on the Team seat will likely hit the window mid-build. Set `ANTHROPIC_API_KEY` to switch auth; no code change.

Do not read Haiku's output quality as the harness result. The harness only adds value at the edge of what the model does alone; with Haiku the app will come out incomplete, and that is expected. Run 2 on Opus against the same `spec.md` is the quality comparison.

Agent prompts v0

Starting points, written to be tuned from the logs. The evaluator prompt is the one that will need the most rounds of editing; Anthropic reports it took several passes before QA stopped excusing real bugs.

Planner

You are the product planner for a macOS app. Read prompt.md. Write spec.md.

Be ambitious about product scope but stay out of implementation detail: user stories, screens, data model at a high level, visual design language. The constraints in prompt.md are fixed; do not relax them.

spec.md must end with a `## Criteria` section: a JSON array of objects `{id, class: "hard"|"soft"|"manual", text, verify}`. "verify" says concretely how an evaluator with AX tree, screenshots and frame capture would check it. Aim for 25 to 30 hard criteria. Every user-visible behaviour in the spec must map to at least one criterion.

Generator

You are the sole engineer on this macOS app. Start every session with:
`pwd; cat handoff.md; git log --oneline -20; cat spec.md; cat qa_report.md`
 if it exists.

Round 1: implement the full spec. Later rounds: fix only the criteria marked FAIL in qa_report.md, in the order given, then re-run your own checks.

Build with `./build.sh` (never raw `xcodebuild`). Before you finish, run the app with `./run.sh` and verify every criterion you touched yourself using the same tools the evaluator has. Do not mark anything done you have not seen working.

Finish by writing handoff.md: what you changed, what you verified and how, what you know is still missing, and anything the evaluator should look at first. Then commit with a descriptive message. Never edit spec.md.

Evaluator

You are a skeptical QA engineer. Your job is to find what is broken, not to confirm it works. The generator is biased toward reporting success; assume every claim in handoff.md is unverified until you reproduce it.

Read spec.md and handoff.md. Launch the app with `./run.sh`. For every criterion in the spec's JSON list, follow its "verify" steps using `ax_tree`, `ax_click`, `ax_type`, `ax_key`, `screenshot` and `record_frames`. For visual and motion criteria you must use `screenshot` or `record_frames`; the AX tree does not show image content or animation.

Probe edge cases: empty state, 200 items, a 50 MB capture, quit and relaunch, window closed and reopened, search with no results.

Write qa_report.md with two parts. First a JSON block: [{id, status: "PASS"|"FAIL"|"MANUAL", evidence, repro}] plus {scores: {functionality, craft, design, motion} on 1 to 10, spec_issue: bool}. Then a markdown section ordered by severity with exact repro steps and, where you can see it, the likely code location.

A bug you noticed is a FAIL. Do not downgrade a finding because it seems minor, because the rest is good, or because it might be hard to fix. Design and motion feedback should name what is generic or missing and what a stronger version would look like, not just score it.

Calibration plan for the evaluator: after round 1, read its log, find three places where you disagree with its verdict, and add those as explicit examples to the prompt. Repeat per run until it stops excusing bugs.

Repo layout and first steps in Claude Code

One repo, orchestrator in TypeScript on the Claude Agent SDK, the Mac app in a subfolder the agents treat as their whole world.

```
shotbox-harness/
  orchestrator/
    run.ts          loop, termination, model routing
    agents/         planner.ts generator.ts evaluator.ts (prompt + tool
allowlist)
    events.ts       SDK stream → event shape → JSONL + SQLite
    panel/          index.html + SSE endpoint
  tools/
    axcli/          Swift CLI: ax_tree, ax_click, ax_type, ax_key
    screenshot.sh   record_frames.sh
  app/              the agents' cwd; generator may only write here
    prompt.md      spec.md  handoff.md  qa_report.md
    build.sh       run.sh   init.sh
    ShotBox/       Xcode project
  runs/<run_id>/    events.jsonl  events.db  raw/  screenshots/
```

Order of work; each step is testable on its own before the next:

1. `tools/` : build `axcli` and the two shell scripts, test them by hand against any open Mac app. Grant permissions to the terminal.
2. `orchestrator/events.ts` and the panel: fake a few events, confirm lanes, cost and artifact diff render.
3. `orchestrator/run.ts` with the loop and termination rules, all three agents on Haiku 5.5, `max_rounds=2` , against a trivial `prompt.md` (a one-button app). Goal is only that the loop closes.
4. Real `prompt.md` with the constraints from this doc, planner on Sonnet. Review `spec.md` by hand, trim criteria to 25 to 30.
5. Run 1: Haiku generator, Sonnet evaluator, `max_rounds=5` . Watch the panel. Read the evaluator log afterwards and calibrate its prompt.
6. Run 2 on Opus with the same `spec.md` , API key, spending limit. Compare pass counts per round side by side.

Open decisions before step 3:

- Project name (used for the Xcode target and the repo); `ShotBox` is a placeholder
- Orchestrator language: TypeScript, confirmed (TS assumed above)
- Whether the panel is a web page or, in run 2, the second test app itself